Лекция12 Объектно-ориентированное представление программных систем. Унифицированный язык моделирования UML

Введение: почему нельзя просто сесть и написать код

Представьте, что вам поручили построить большой торговый центр. Вы же не начнёте с того, что привезёте кирпичи и скажете рабочим: «Кладите как-нибудь, потом разберёмся». Сначала архитектор рисует чертежи: где будут магазины, как пройдут коммуникации, где входы и выходы, как эскалаторы соединят этажи.

С программными системами — та же история. Чем крупнее проект, тем важнее сначала **описать** систему, а потом **делать**. И делать это так, чтобы и заказчик понял, и программист не запутался, и тестировщик знал, что проверять.

Именно для этого существует **UML** — унифицированный язык моделирования. Это не язык программирования. На нём нельзя написать код. На нём можно **нарисовать** систему так, чтобы её поняли все участники проекта.

Но прежде чем говорить о UML, нужно разобраться, что значит **объектно-ориентированное представление** системы.

Объектно-ориентированное представление: смотрим на мир через объекты

Функциональный взгляд против объектного

Раньше программисты думали так: «Есть задача. Её нужно разбить на функции. Одна функция принимает данные, другая их обрабатывает, третья выводит результат». Это **функциональный подход**. Данные — отдельно, функции — отдельно.

Но у такого взгляда есть проблема. Представьте банк. Есть данные: клиенты, счета, транзакции. Есть функции: открыть счёт, перевести деньги, закрыть счёт. Всё вроде логично. Но когда система разрастается, оказывается, что функция «перевести деньги» должна знать всё о счетах, клиентах, лимитах, комиссиях. А функция «закрыть счёт» — тоже. И каждая новая функция тащит за собой всё больше данных. Система становится запутанной.

Объектный подход предлагает другой взгляд. Мир состоит не из функций, а из **объектов**. Клиент — объект. Счёт — объект. Транзакция — объект. У каждого объекта есть:

- **Состояние** — то, что объект «знает» о себе. У клиента — имя, паспорт, телефон. У счёта — номер, баланс, владелец.
- **Поведение** — то, что объект «умеет делать». Клиент может изменить телефон. Счёт может принять деньги, списать деньги, показать баланс.
- **Индивидуальность** — каждый объект отличается от других. Даже два клиента с одинаковым именем — разные объекты.

Теперь функция «перевести деньги» — это не отдельная сущность. Это **метод** объекта «Счёт». Он работает с данными, которые принадлежат этому объекту. Ничего лишнего не тащит.

Четыре кита объектного подхода

Объектно-ориентированное представление строится на четырёх принципах.

Абстрагирование. Мы не описываем объект целиком, со всеми его физическими свойствами. Мы выделяем только то, что важно для системы. Для банка клиент — это ФИО, паспорт, телефон. А его любимый цвет или размер обуви — не важны. Абстракция отсекает лишнее.

Инкапсуляция. Данные объекта скрыты от внешнего мира. Клиент не может напрямую изменить баланс счёта — он должен вызвать метод «пополнить». Внутри метода проверяются правила: сумма положительная, счёт не заблокирован. Инкапсуляция защищает данные от случайной порчи.

Наследование. Новый класс можно построить на основе существующего. Например, есть класс «Счёт». От него можно унаследовать «Сберегательный счёт» и «Кредитный счёт». Они получат все свойства и методы родителя, но смогут добавить свои. Наследование экономит код и делает структуру логичной.

Полиморфизм. Один и тот же метод может работать по-разному для разных классов. Метод «рассчитать проценты» для сберегательного счёта работает иначе, чем для кредитного. Но вызывается одинаково. Это делает систему гибкой.

Зачем это всё нужно

Объектный подход хорош тем, что **модель системы становится похожа на реальный мир**. Клиент — это клиент. Счёт — это счёт. Заказ — это заказ. Не нужно держать в голове «а вот тут у нас функция, которая работает с тремя таблицами, но не со всеми полями».

Кроме того, объектная модель **устойчива к изменениям**. Если завтра банк захочет добавить новый тип счёта, достаточно создать новый класс-наследник. Остальная система не сломается.

Именно поэтому UML, как язык моделирования, построен вокруг объектов.

UML: что это и зачем

***UML** (Unified Modeling Language) — унифицированный язык моделирования. Это **графический язык** для описания, визуализации, проектирования и документирования программных систем.*

UML — не язык программирования. На нём нельзя написать код, который скомпилируется. UML — это способ **нарисовать** систему. Как чертёж для архитектора. Как ноты для композитора.

UML был создан в середине 1990-х годов тремя специалистами: Гради Бучем, Джеймсом Рамбо и Иваром Якобсоном. Они объединили свои подходы в один язык, который в 1997 году был принят как стандарт консорциумом OMG (Object Management Group). С тех пор UML развивается, и сейчас актуальна версия UML 2.x.

Зачем нужен UML

Представьте, что заказчик говорит разработчику: «Хочу, чтобы клиент мог оформить заказ». Разработчик кивает. А потом выясняется, что заказчик имел в виду одно, а разработчик сделал другое. Знакомая ситуация?

UML решает эту проблему. Он даёт **единый язык** для всех участников проекта: заказчиков, аналитиков, разработчиков, тестировщиков. Все смотрят на одну диаграмму и понимают её одинаково.

Кроме того, UML помогает:

- **Увидеть систему целиком** до того, как написан код.

- **Найти ошибки** на ранних этапах, когда их исправление дёшево.
- **Разделить работу** между командами.
- **Задокументировать** систему для будущих поколений.

Категории диаграмм UML

UML — это не одна диаграмма, а **набор диаграмм**. Каждая показывает свой «срез» системы. Все диаграммы делятся на две большие группы:

Структурные диаграммы — показывают, **из чего состоит** система. Что есть в системе, как части связаны между собой. Это статический взгляд.

Поведенческие диаграммы — показывают, **как система работает**. Что происходит, когда пользователь нажимает кнопку, как система реагирует на события. Это динамический взгляд.

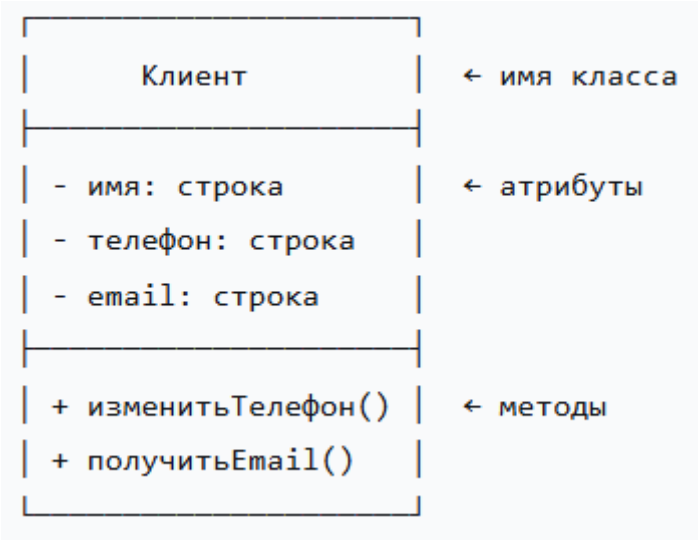
Рассмотрим основные диаграммы.

Структурные диаграммы

Диаграмма классов

Диаграмма классов *— самая важная структурная диаграмма UML. Она показывает классы системы, их атрибуты, методы и связи между классами.*

Класс изображается прямоугольником, разделённым на три части:



Знаки перед атрибутами и методами обозначают **видимость**:

- + — публичный (доступен всем)
- - — приватный (доступен только внутри класса)
- # — защищённый (доступен классу и наследникам)

Связи между классами изображаются линиями разного вида:

Ассоциация — обычная линия. Показывает, что классы связаны. Например, «Клиент» связан с «Заказом».

Агрегация — линия с пустым ромбом. Показывает, что один класс содержит другой, но они могут существовать отдельно. Например, «Отдел» содержит «Сотрудников», но если отдел закроют, сотрудники останутся.

Композиция — линия с закрашенным ромбом. Более сильная связь. Например, «Дом» состоит из «Комнат». Если дом разрушить, комнаты исчезнут.

Наследование — линия со стрелкой в виде треугольника. Показывает, что один класс наследует другой.

Пример. Для банковской системы диаграмма классов может выглядеть так:



Здесь видно: у клиента может быть много счетов (1 ко многим). Сберегательный счёт наследует обычный счёт и добавляет свою ставку.

4.2. Диаграмма объектов

Похожа на диаграмму классов, но показывает **конкретные объекты** в конкретный момент времени. Если диаграмма классов — это чертёж, то диаграмма объектов — это фотография.

Например, диаграмма классов говорит: «У клиента есть счёт». А диаграмма объектов говорит: «У Иванова Ивана есть счёт №12345 с балансом 5000 рублей».

4.3. Диаграмма компонентов

Показывает, **из каких программных модулей** состоит система и как они связаны. Компоненты — это более крупные единицы, чем классы. Например, «Модуль авторизации», «Модуль отчётов», «Модуль работы с БД».

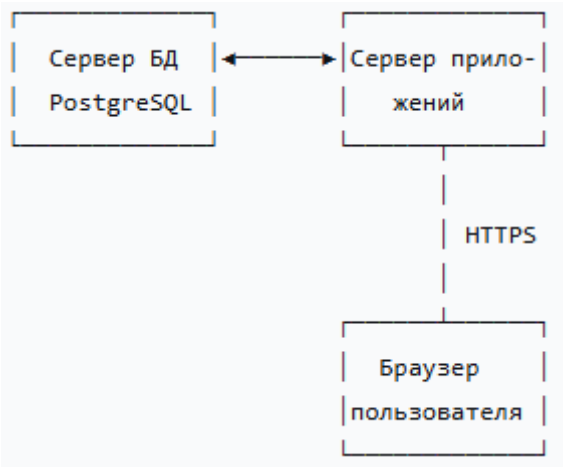
Изображаются прямоугольниками с символом компонента. Связи показывают, кто кого использует.

4.4. Диаграмма развёртывания

Показывает, **на каком оборудовании** работает система. Какие серверы, какие компьютеры пользователей, как они соединены сетью.

Изображается в виде трёхмерных кубов (узлов), соединённых линиями. Внутри узлов могут быть указаны компоненты.

Пример. Для системы интернет-магазина:



5. Поведенческие диаграммы

5.1. Диаграмма вариантов использования

Диаграмма вариантов использования (Use Case Diagram) — показывает, **что система делает для пользователя**. Это взгляд с точки зрения пользователя.

Основные элементы:

Актёр — человечек. Это роль, которую играет пользователь или внешняя система. Например, «Клиент», «Администратор», «Платёжная система».

Вариант использования — овал. Это функция, которую система предоставляет. Например, «Оформить заказ», «Просмотреть историю».

Связи — линии между актёрами и вариантами использования.

Пример. Для системы туроператора:

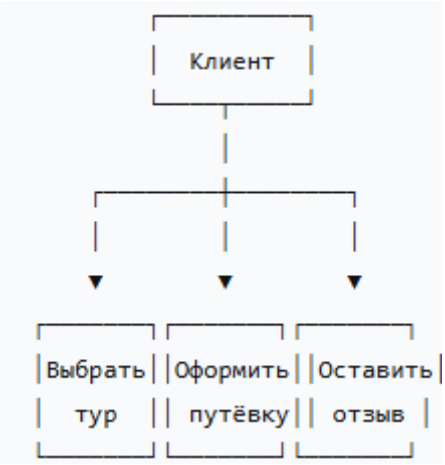


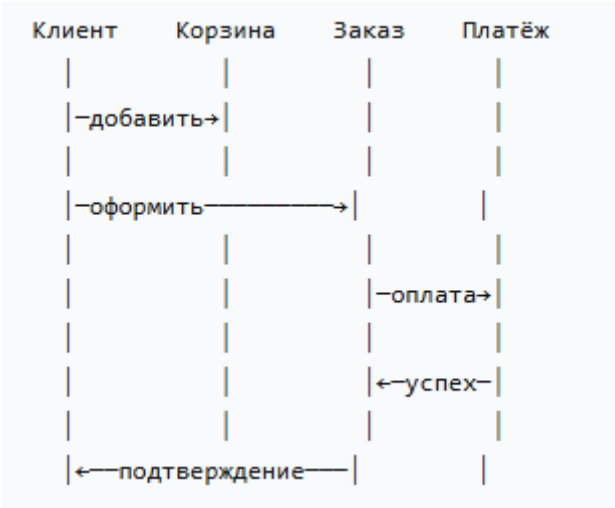
Диаграмма вариантов использования — это первый документ, который показывает заказчику, что система будет делать. Она не углубляется в детали реализации — только функции.

5.2. Диаграмма последовательности

Диаграмма последовательности (Sequence Diagram) — показывает, **как объекты обмениваются сообщениями во времени**.

Основные элементы: объекты (прямоугольники сверху), линии жизни (пунктирные линии вниз), сообщения (стрелки между линиями жизни).

Пример. Оформление заказа:



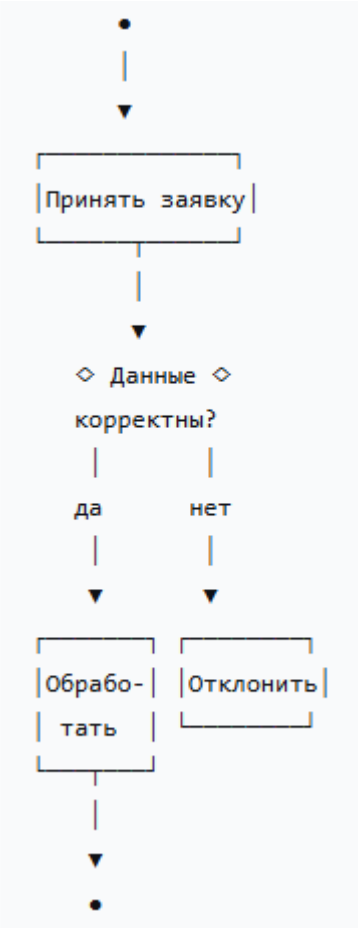
Здесь видно: клиент сначала добавляет товар в корзину, потом оформляет заказ, система отправляет запрос на оплату, получает подтверждение и возвращает клиенту результат.

5.3. Диаграмма деятельности

Диаграмма деятельности (Activity Diagram) — по сути, блок-схема алгоритма. Показывает последовательность действий, условия, параллельные ветви.

Основные элементы: действия (скруглённые прямоугольники), решения (ромбы), начало и конец (круги), параллельные ветви (жирные линии).

Пример. Обработка заявки:

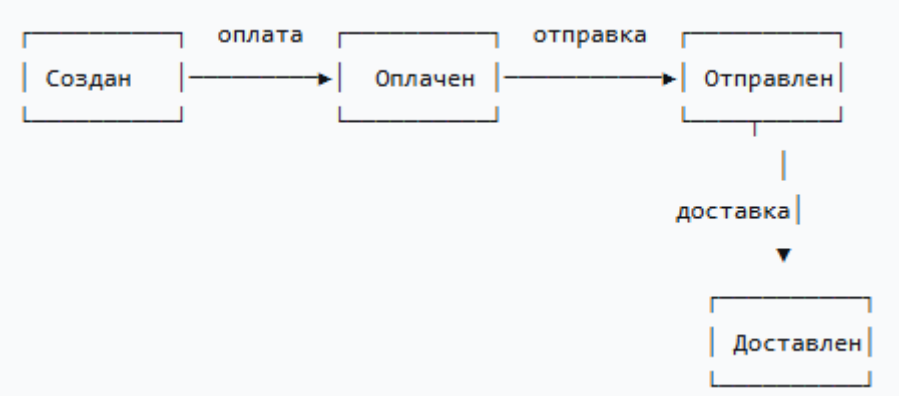


5.4. Диаграмма состояний

Диаграмма состояний (State Diagram) — показывает, **как объект меняет своё состояние** под воздействием событий.

Основные элементы: состояния (скруглённые прямоугольники), переходы (стрелки с подписями событий).

Пример. Состояния заказа:



Заказ не может перейти из «Создан» сразу в «Доставлен» — он должен пройти через все промежуточные состояния. Диаграмма состояний это показывает наглядно.

5.5. Диаграмма коммуникации

Похожа на диаграмму последовательности, но акцент — не на времени, а на **связях между объектами**. Показывает, кто с кем взаимодействует, а не в каком порядке.

При построении диаграммы сфокусируемся на передаче именно тех данных или сообщений (выявленных в ходе обсуждения со стейкхолдерами), которые имеют отношение к нашей задаче.

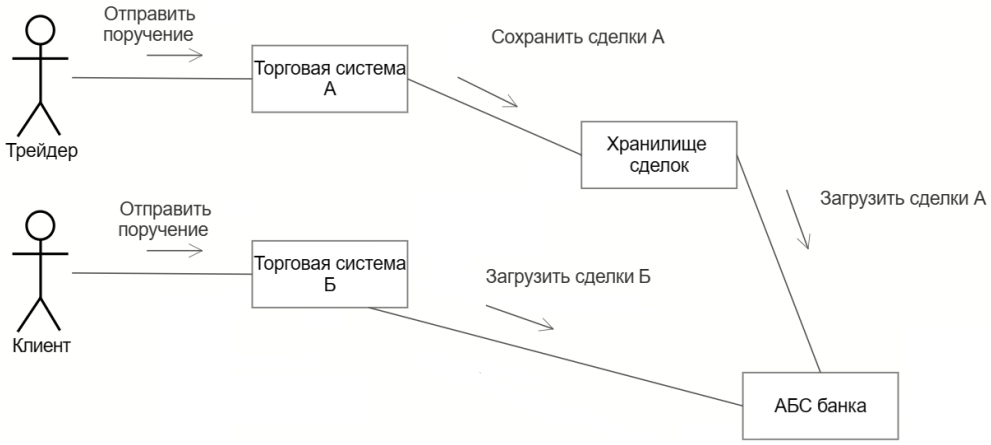


Диаграмма коммуникации в первом приближении

Выглядит неплохо, но есть пока вопросы, как то: какой компонент будет отвечать за управление лимитами, на какие существующие операции внутри компонентов мы повлияем своей доработкой, а какие операции будут влиять на нас?

По мере погружения в проблематику, выявления заинтересованных сторон, изучения документации и поиска вариантов решения мы уточняем диаграмму и она приобретает следующий вид.



Диаграмма коммуникации, обогащённая новыми деталями и идеями

В последней представленной версии, как можно видеть, был изменён принцип загрузки сделок из Торговой системы Б, а также появились новые детали.

На этом моменте процесс уточнения диаграммы предлагаю остановить и сосредоточиться на значимых для работы аналитика фактах.

Во-первых, связи и сообщения на диаграмме коммуникации крайне полезны; они, как пункты чек-листа, помогают в выявлении функциональных требований. Объясняется это тем, что каждое сообщение на схеме визуализирует существование функциональной зависимости, а значит добавление на схему нового объекта, сообщения или связи потенциально может повлиять на все ранее сложившиеся зависимости.

Во-вторых, выразительные средства диаграммы коммуникации позволяют в явном виде показать не только внешние взаимодействия, но и важные для нас операции, выполняемые внутри систем. Это изображается с помощью петель — рефлексивных связей, которые обеспечивают передачу сообщений от объекта самому себе. Петли существенно повышают наглядность и практическую полезность выбранного средства визуализации, поэтому я бы рекомендовал не избегать возможности применять их на своих диаграммах.

В-третьих, несмотря на то, что на последней диаграмме были пронумерованы сообщения, надо понимать, что чёткой хронологии здесь нет. И это обстоятельство является достоинством, а не недостатком. В отличие от диаграммы последовательности, которая в общем случае предполагает строгую очерёдность выполнения шагов, в рассматриваемом кейсе многие события развиваются во времени независимо и параллельно. Соответственно, если бы мы пытались это визуализировать с помощью диаграммы последовательности, то получился бы либо набор из нескольких диаграмм, описывающих отдельные цепочки взаимодействия, существующие в рамках схемы автоматизации (а значит увидеть картину целиком было бы затруднительно), либо, в худшем случае, у нас появилась бы единая диаграмма с «мешаниной», которая привела бы к неправильному представлению ситуации в целом, что в конечном итоге могло бы привести к ошибкам в проектировании.

6. Пример: моделирование системы «Библиотека» на UML

Соберём всё вместе. Представим, что мы проектируем систему для библиотеки.

Диаграмма вариантов использования. Актёры: «Читатель», «Библиотекарь». Варианты использования: «Взять книгу», «Вернуть книгу», «Продлить срок», «Найти книгу», «Добавить книгу».

Диаграмма классов. Классы: «Читатель», «Книга», «Экземпляр книги», «Выдача». Связи: читатель может иметь много выдач, каждая выдача связана с одним экземпляром.

Диаграмма последовательности. Читатель приходит, библиотекарь сканирует книгу, система проверяет, есть ли книга в наличии, создаёт запись о выдаче, возвращает подтверждение.

Диаграмма состояний. Экземпляр книги может быть в состояниях: «В библиотеке», «Выдан», «На ремонте», «Списана».

Так одна и та же система описывается с разных сторон. Каждая диаграмма отвечает на свой вопрос. Вместе они дают полную картину.

Логическая модель - модель предметной области. И хотя предметная область библиотеки не является чем-то сложным, в процессе проектирования требуется

Выделение классов

Метод именных групп

Выпишем существительные из текста задачи. Сопоставим каждому чуществительному возможный класс.

Существительное	Место в модели
система	абстрактно, не имеет значения
библиотека	система целиком, не класс
книга	Класс
автор	Атрибут книги. Как класс хранил бы мало информации
издательство	Атрибут книги. Как класс хранил бы мало информации
тематика	см. Тематический каталог
наличие	не класс, но требование хранить состояние всех книг (доступна/на руках)
сайт	Деталь реализации. См. раздел "Обновленная структурная модель"
учет	Функциональное требование (отглагольное сущ.)
абонемент	Отдел библиотеки. В нашей задаче не имеет смысла
срок окончания	атрибут книги, взятой на руки
полнотекстовый поиск	Деталь реализации. Подразумевается поиск в атрибутах класса Книга
тематический каталог	Иерархия тем. Нужен класс для реализации
Литература	синоним "Книга". Сливаем вместе
читатель	Нужен класс для представления пользователя в системе
на дом	Не класс, но состояние книги
срок 30 дней	атрибут книги, взятой на руки

Существительное	Место в модели
возврат документов	От глагола "возвратить". Не класс
денежная компенсация	Класс не нужен. Скорее от глагола "компенсировать"
Продление	От глагола "продлить". Не класс
истечения основного срока	От "истекать". Похоже на временной сигнал
телефон	Класс не нужен. Функциональное требование
интернет	Класс не нужен. Функциональное требование

Выпишем глаголы и отглагольные существительные. Сопоставим галголам операции классов или Варианты использования

Глагол	Место в Модели
искать	Вариант использования "Искать книгу"
узнавать о наличии	Часть ВИ "Искать книгу"
вести учет	ВИ "Выдать книгу", "Принять книгу"
извещать	ВИ "Известить о сроке возврата"
выдается на срок	Часть требования "вести учет"
возврата (срок)	часть других ВИ
уплатить	ВИ "Оштрафовать"
Продление	ВИ "Продлить"

Дополнительные соображения

Очевидно, чтобы вести учет выданных книг, нужн хранить *какая книга* выдана и *кому* выдана. Нам нужен класс для хранения информации о читателе. Назовем его `Reader`.

Reader	
хранит ФИО	Book
хранит email	
хранит телефон	

Также нам нужен класс, хранящий информацию о книге `Book`. Это не указано явно в тексте задачи, но в библиотеке много одинаковых книг. Что должен моделировать объект класса книга: издание или отдельный экземпляр? В нашей задаче не нужно отличать отдельные экземпляры книг. Это даже вредно: если два читателя по ошибке поменяются экземплярами, процесс возврата будет усложнен. Итак книга - это издание. Чтобы хранить информацию о доступности и отдельных экземплярах, введем атрибуты "Количество кземпляров", "Количество доступных экземпляров".

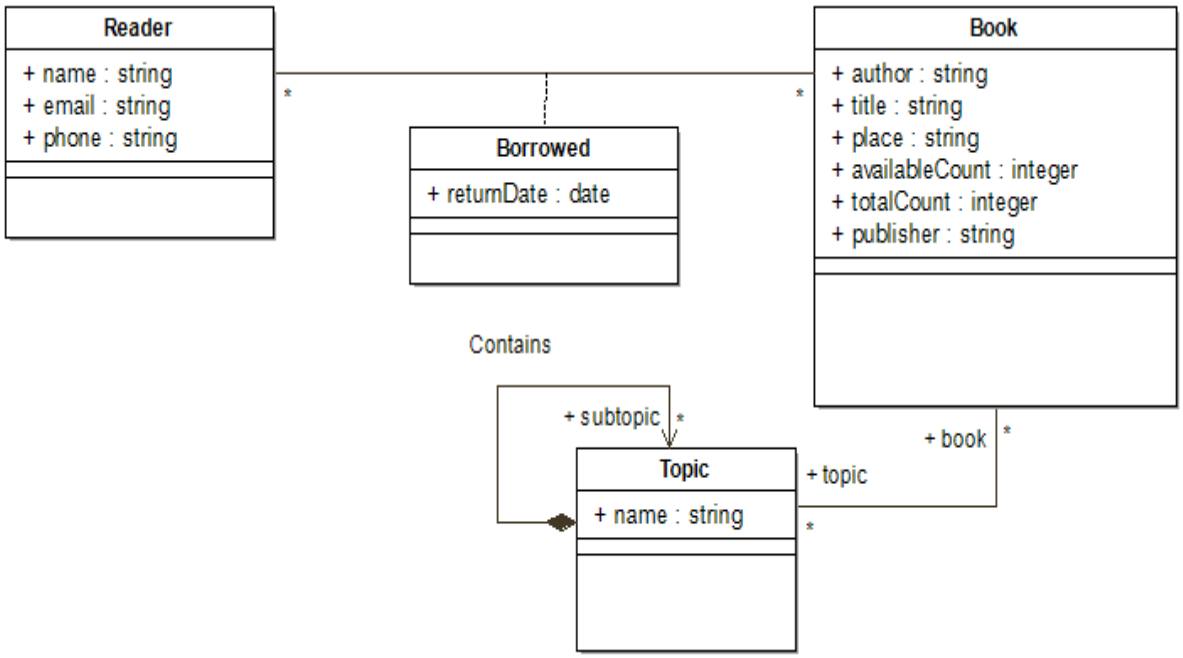
Book	
хранит Название	Topic
хранит Автора	
хранит количество экземпляров	
хранит количество доступных экз-ов	

Книга, выданная читателю на руки будет моделироваться ассоциацией `Borrowed` ("на руках"). Срок выдачи удобно сделать атрибутом этой ассоциации, будем использовать класс ассоциации.
Тематический каталог представляет собой дерево категорий `Topic`.

Модель использования

Искать книгу может как `Читатель`, так и `Библиотекарь`. Естественно для этого ввести действующее лицо (актора) `Искатель`, обобщающее акторов, которым доступен поиск. Тот факт, что библиотекарю в результате поиска нужно отобразить служебную информацию (адрес шкафа, полки, где размещены экземпляры книги), можно игнорировать в логической модели или перенести в ВИ `Выдать книгу`.
Обратите внимание, не `Читатель` берет книгу, а `Библиотекарь` выдаёт. Именно обязанность библиотекаря зарегистрировать факт выдачи книги. Аналогично в ВИ `Принять книгу` - взаимодействие с системой в стенах библиотеки ведёт `Библиотекарь`.
Также обратите внимание, что продление через сайт инициируется `Читателем`, а продление по телефону инициируется `Библиотекарем`.

Структурная модель



Обратите внимание, множественность `*` у ассоциации `Borrowed` со стороны `Reader` означает, что конкретная книга может быть свзяана выдана разным читателям, ведь объект книги это множество бумажных экземпляров.

Реализация вариантов использования

Для некоторых наиболее важных ВИ построим текстовое описание и модель поведения.

ВИ "Искать по тексту"

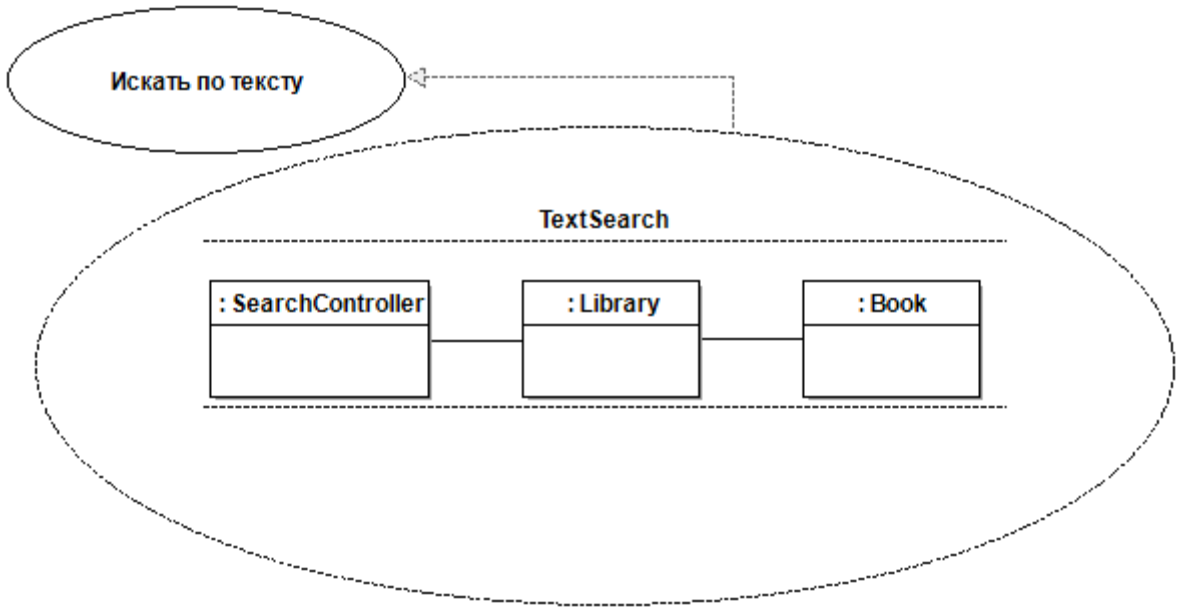
Читатель или библиотекарь по текстовому запросу ищут конкретную книгу в библиотеке.

Текстовая реализация

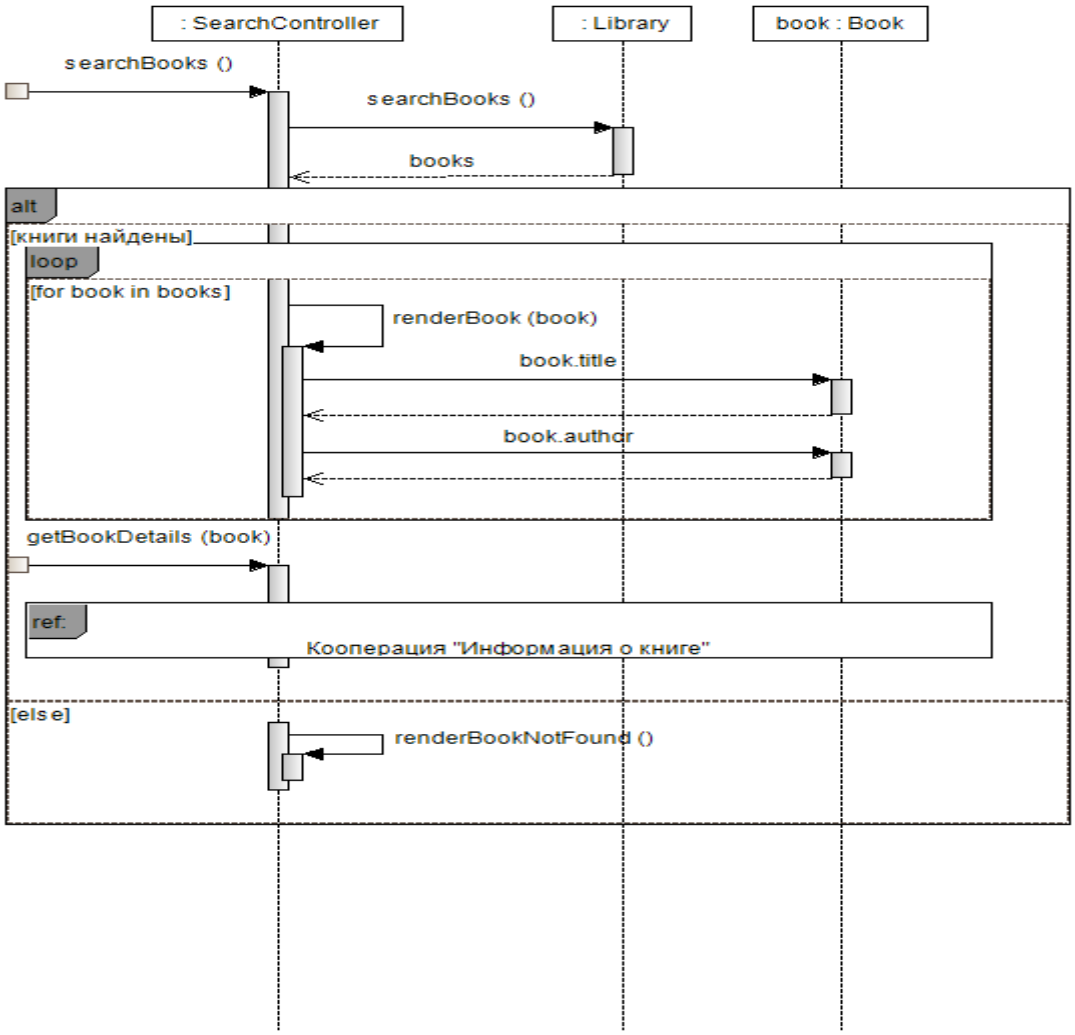
- Акторы: Искатель
- Цель: Получить список книг
- Предусловия: -
- Постусловия: Каждая найденная книга удовлетворяет тексту запроса
- Основной сценарий:
    1. Актор вводит текст запроса.
    2. Актор видит книги `Book`, автор или название которых содержит текст запроса.
    3. Актор выбирает искомую книгу.
- Альтернативный сценарий:
    1. Актор видит сообщение, что книги не найдены.

Модель поведения

Для того, чтобы моделировать поиск книг, естественно ввести операцию найти книги `searchBooks( request:string ) : Book[*]`. Какому классу назначить эту ответственность? Добавим класс библиотека `Library`. Согласно паттерну GRASP *Pure Fabrication* он не соответствует объектам предметной области, а нужен лишь для выполнения некоторых обязанностей. Однако, удобно думать о `Library` как о центральном объекте системы, который "хранит" все прочие объекты: читателей, книги, тематический каталог.
Откуда приходят запросы? Как отображаются результаты? Эти вопросы детально прорабатываются в модели реализации. Однако на данном этапе удобно ввести класс `SearcherController`, согласно паттерну GRASP *Controller*. Это будет точка входа в логическую модель.
Кооперация, реализующая поведение ВИ "Искать по тексту" будет выглядеть так:



Основной сценарий взаимодействия представим в виде диаграммы последовательности:



Обратите внимание, сейчас нам не важно, как именно работает `searchBooks`. Это можнет быть полный перебор объектов с сопоставлением текста или запрос в базу данных с помощью поискового индекса. Оставим это модели реализации. Также мы не детализируем, как именно происходит рендеринг книги для пользователя - опять же, на усмотрение реализации.

Получение детальной информации по книге не является частью сценария, но будет полезно отобразить на диаграмме свзязь между вариантами использования.

ВИ "Искать в каталоге"

Читатель или библиотекарь ищут конкретную книгу, зная тематику.

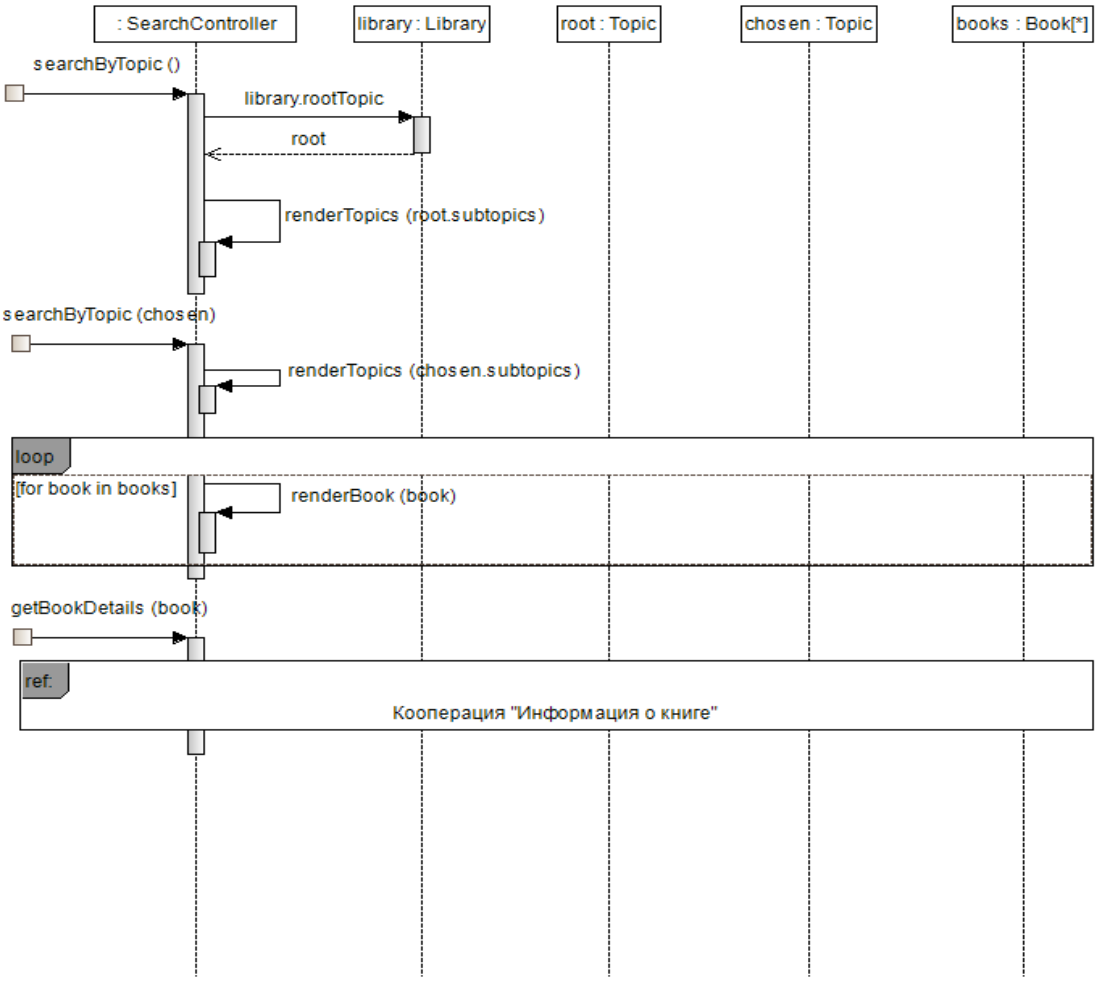
Текстовая реализация

- Акторы: Искатель
- Цель: Получить список книг
- Предусловия: -
- Постусловия: Каждая найденная книга удовлетворяет искомой тематике
- Основной сценарий:
    1. Актор выбирает тему из списка.

2. Актору видит список дочерних тем и несколько книг с заданной темой.
3. Актор выбирает дочернюю тему.
4. Актору видит список дочерних тем (при наличии) и первые 10 книг с заданной темой.
5. Актор выбирает искомую книгу.

Модель поведения

Кооперация, реализующая данный ВИ
содержит `SearchController`, `Library`, `Topic` (кратность 1..*), и
книги `Book` *(кратность*).
Покажем поведение на примере двух выбранных тем:



ВИ "Выдать книгу"

Безусловно, ключевой ВИ системы. Библиотекарь выдаёт книгу читателю и вносит об этом информацию в систему.

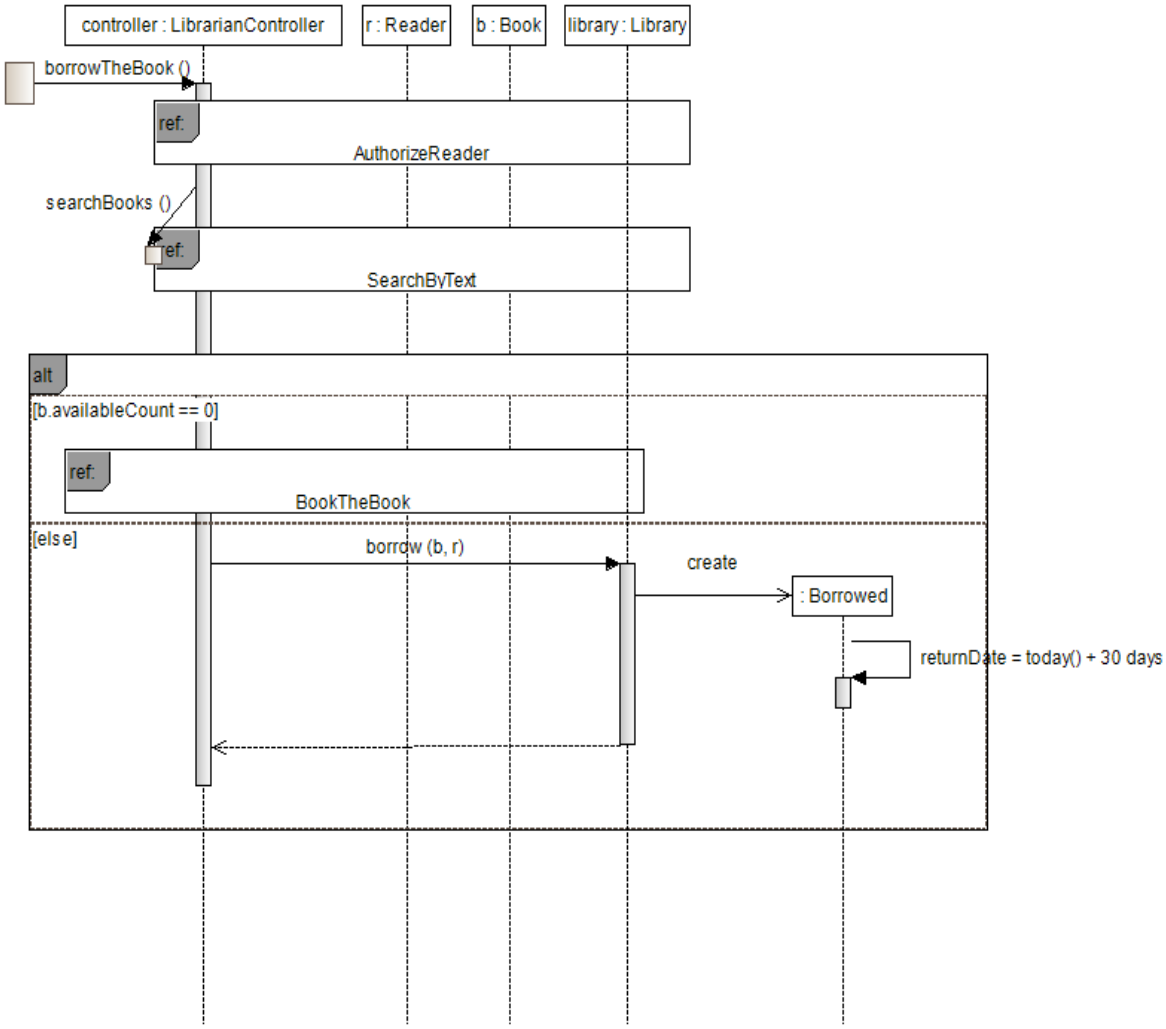
Тут можно было бы моделировать отказ выдать книгу при условии, что у читателя слишком много книг на руках, но для простоты мы опустим это поведение, к тому же этого нет в тексте постановки задачи.

Текстовая реализация

- Акторы: Библиотекарь
- Цель: Зарегистрировать выданную читателю книгу
- Предусловия: -
- Постусловия: -
- Основной сценарий:
    1. ВИ "Авторизовать читателя"
    2. ВИ "Искать книгу"
    3. Если книга свободна, Библиотекарь назначает книгу пользователю со стандартным сроком возврата.
- Альтернативный сценарий:
    1. Если книга занята, предложить Читателю забронировать книгу на другую дату.
    2. ВИ "Забронировать книгу"

Модель поведения

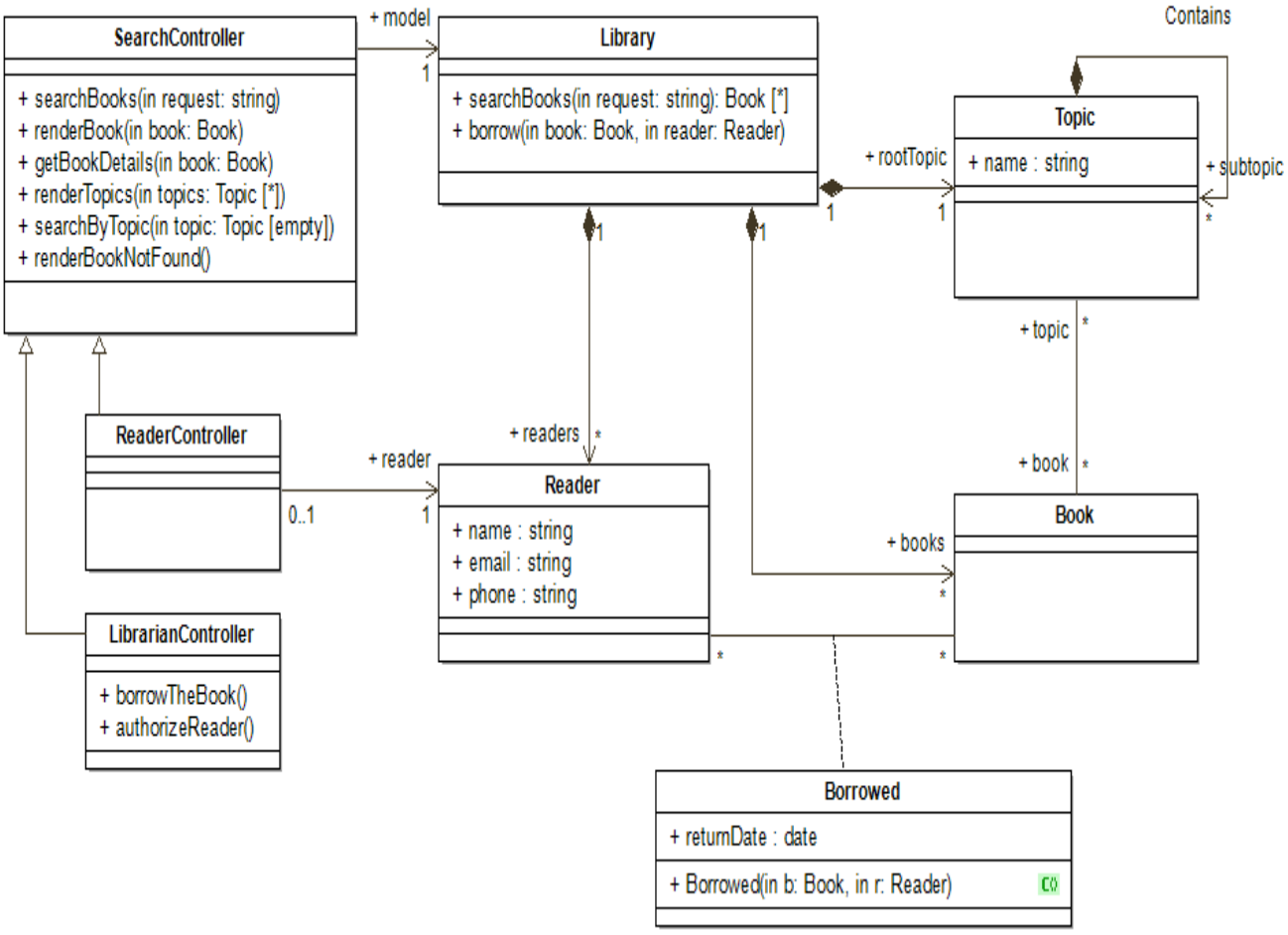
Кооперация, реализующая данный ВИ содержит `LibrarianController`, `Library`, `Book`, `Reader`, `Borrowed`.



Обратите внимание, вызов searchBooks() в данном сценарии делает сам `controller`. На диаграмме последовательности ВИ "Искать по тексту" видно, что с этого вызова начинается последовательность. Это демонстрирует применение шлюзов (gates) и найденных сообщений.

Обновленная структурная модель

В результате анализа вариантов использования в структурную модель были включены классы-
контроллеры `SearchController`, `ReaderController`, `LibrarianController`, а также класс самой библиотеки `Library`, представляющий владение остальными объектами системы.
В существующие классы были добавлены методы. В частности, появился конструктор класса-ассоциации `Borrowed`, заполняющий дату возврата книги.
Обратите внимание! Класс `Reader` и класс `ReaderController` - не одно и то же. `Reader` - это то, как библиотека видит читателя. `ReaderController` - это то, как читатель видит библиотеку, собственно это интерфейс библиотеки для читателя.



Собственное поведение классов

7. Как читать UML-диаграммы: практические советы

Смотрите на цель диаграммы. Структурные диаграммы показывают «что есть». Поведенческие — «что происходит». Не путайте.

Не пытайтесь запихнуть всё в одну диаграмму. UML специально разделён на виды, чтобы каждый показывал свой аспект. Если диаграмма стала нечитаемой — разбейте её.

Используйте инструменты. Рисовать UML вручную на бумаге можно, но неудобно. Есть специальные программы: draw.io, PlantUML, StarUML, Enterprise Architect. Они позволяют быстро строить диаграммы и поддерживать их в актуальном состоянии.

Не усложняйте. UML — мощный язык, но не все его элементы нужны в каждом проекте. Начните с диаграммы классов и вариантов использования. Остальное добавляйте по мере необходимости.

8. Итог

- **Объектно-ориентированное представление** рассматривает систему как набор объектов с состоянием, поведением и индивидуальностью. Опирается на четыре принципа: абстрагирование, инкапсуляция, наследование, полиморфизм.
- **UML** — унифицированный язык моделирования. Это графический язык для описания систем. Не язык программирования.
- **Структурные диаграммы** (классов, объектов, компонентов, развёртывания) показывают, из чего состоит система.
- **Поведенческие диаграммы** (вариантов использования, последовательности, деятельности, состояний, коммуникации) показывают, как система работает.
- Каждая диаграмма отвечает на свой вопрос. Вместе они дают полную картину системы.
- UML помогает договориться заказчику и разработчику, найти ошибки на ранних этапах и задокументировать систему.

Научиться читать и строить UML-диаграммы — значит научиться **думать о системе**, а не только писать код. Это навык, который отличает программиста от проектировщика. И это первый шаг к тому, чтобы создавать сложные системы, которые не разваливаются через месяц после запуска.

Контрольные вопросы

1. Что такое объектно-ориентированное представление системы?
2. Назовите четыре принципа объектно-ориентированного подхода и объясните каждый.
3. Что такое UML и зачем он нужен?
4. Чем отличаются структурные диаграммы UML от поведенческих?
5. Что показывает диаграмма классов?
6. Какие типы связей существуют между классами? Чем агрегация отличается от композиции?
7. Для чего используется диаграмма вариантов использования?
8. Что показывает диаграмма последовательности?
9. Чем диаграмма деятельности отличается от диаграммы состояний?
10. Какую диаграмму UML выбрать, чтобы показать, на каком оборудовании работает система?
11. Что такое актёр на диаграмме вариантов использования?
12. Можно ли на UML написать программу?

13. Зачем нужно моделировать систему до написания кода?
14. Какие инструменты используются для построения UML-диаграмм?
15. Приведите пример системы и опишите её с помощью двух-трёх диаграмм UML.